

Rapport de Projet de Fin d'Études

Développement d'un système de collecte de données en temps réel et de gestion des alertes pour les machines en salle de réveil

Zaibi Racha

June 10, 2025

Contents

Introduction Générale	3
1 Cadre Général de Projet	4
1.1 Présentation de l'organisme d'accueil	4
1.2 Présentation de projet	4
1.2.1 Objectifs Principaux	4
1.2.2 Portée et Périmètre	5
1.3 Étude de l'existant	5
1.3.1 Présentation de l'étude de l'existant	5
1.3.2 Critiques de l'existant	5
1.4 Solution proposée	5
1.4.1 Collecte de données	5
1.4.2 Gestion des alertes basée sur des règles	5
1.5 Objectifs	6
2 Méthodologie adoptée	7
2.1 Choix de la méthodologie	7
2.1.1 Comparaison des méthodologies courantes	7
2.1.2 Rationale détaillé pour le choix de Scrum	7
2.2 Présentation de Scrum	7
2.2.1 Artefacts Scrum et leur application	8
2.2.2 Événements Scrum et leur implémentation	9
2.3 Les rôles Scrum	9
2.4 Gestion des risques avec Scrum	10
2.5 Engagement des parties prenantes dans Scrum	10
2.6 Outils de support à Scrum	11
2.7 Représentation visuelle du processus Scrum	11
3 Étude architecturale du projet	12
3.1 Introduction	12
3.2 Architecture globale	12
3.3 Diagrammes d'architecture	13
3.3.1 Diagramme de déploiement	13
3.3.2 Diagramme de composants	14
3.4 Justification des choix technologiques	14
3.5 Flux de données	15
3.6 Architecture physique et fonctionnement	16
3.6.1 Composants matériels	16
3.6.2 Fonctionnement	17
3.6.3 Spécifications logicielles	17
3.6.4 Flux de communication	17
3.7 Justification de l'architecture	18
3.8 Conclusion	18

4	Sprint 0	19
4.1	Introduction	19
4.2	Capture des besoins	19
4.2.1	Identification des acteurs	19
4.2.2	Besoins fonctionnels et non fonctionnels	19
4.2.3	Diagramme des cas d'utilisation global	20
4.3	Pilotage du projet avec Scrum	21
4.3.1	Planification des sprints	23
4.4	Environnement de travail	25
4.4.1	Environnement matériel	25
4.4.2	Environnement de développement	25
4.4.3	Environnement logiciel	26

Introduction Générale

Dans un contexte médical en constante évolution, l'optimisation de la surveillance des patients en salle de réveil constitue un enjeu crucial pour assurer des soins de qualité et réactifs. En Tunisie, de nombreux établissements de santé dépendent encore de méthodes manuelles et de systèmes centralisés limités, ce qui entraîne des délais dans la détection des situations critiques. Par exemple, les infirmiers doivent vérifier physiquement les constantes vitales des patients, ce qui augmente le risque d'erreurs humaines et la charge de travail, particulièrement dans les hôpitaux à ressources limitées où les équipements modernes sont rares.

Ce projet, réalisé au sein du SmartLab de la Faculté de Médecine de Monastir, s'inscrit dans une démarche d'innovation visant à moderniser la surveillance post-opératoire. Il propose un système de collecte de données en temps réel et de gestion des alertes, combinant l'extraction de données via la reconnaissance optique de caractères (OCR) pour les moniteurs. L'objectif principal est d'améliorer la réactivité du personnel médical face aux situations critiques, tout en réduisant les interventions manuelles et les erreurs.

Signification du projet

Ce système répond à plusieurs besoins critiques dans le domaine médical :

- **Pour les patients :** Une détection rapide des anomalies vitales (fréquence cardiaque, pression artérielle, saturation en oxygène) permet une intervention immédiate, améliorant les résultats cliniques.
- **Pour le personnel médical :** Les notifications en temps réel via mobile et web réduisent la nécessité de déplacements constants, permettant aux infirmiers et médecins de se concentrer sur les cas prioritaires.
- **Pour les hôpitaux :** L'automatisation de la surveillance améliore l'efficacité opérationnelle et réduit les coûts liés aux erreurs humaines ou aux retards d'intervention.

En outre, la solution est conçue pour être évolutive, adaptable à différents types de moniteurs et conforme aux réglementations médicales en matière de sécurité des données.

Chapter 1

Cadre Général de Projet

1.1 Présentation de l'organisme d'accueil

Le projet a été mené dans **SmartLab** de la **Faculté de médecine de Monastir (FMM)**. La Faculté de médecine de Monastir est une faculté tunisienne créée le 15 août 1980, dédiée à l'enseignement médical et à la recherche scientifique. Reconnue pour son engagement envers l'excellence, FMM a obtenu une accréditation internationale et la certification **ISO 21**, soulignant son rôle de leader dans l'éducation médicale et la recherche.

SmartLab est une **Unité de Service Commun et de Recherche (USCR)** de pointe associée à FMM, qui se concentre sur la promotion de l'innovation dans les sciences de la santé par le biais d'efforts de collaboration entre le secteur des soins de santé et les start-ups. En intégrant la recherche, la technologie et l'entrepreneuriat, SmartLab vise à faire progresser les soins médicaux et à créer des solutions adaptées aux défis de la santé moderne. Il sert de plaque tournante pour une collaboration interdisciplinaire, en tirant parti de l'expertise pour combler les lacunes entre la recherche et les applications réelles, contribuant ainsi à l'évolution d'une nouvelle ère en médecine.



Figure 1.1: Logo FMM



Figure 1.2: Logo SmartLab

1.2 Présentation de projet

Le projet, intitulé Développement d'un système de collecte de données en temps réel et de gestion des alertes pour les machines en salle de réveil, s'inscrit dans une démarche d'innovation technologique au service des pratiques médicales. Il vise à transformer la surveillance post-opératoire par une approche intégrée combinant **IoT**, **traitement de données** et **notifications en temps réel**.

1.2.1 Objectifs Principaux

- **Acquisition des données :**
 - Collecte automatisée des paramètres vitaux (fréquence cardiaque, pression artérielle, SpO_2)
- **Gestion des alertes :**
 - Notifications basées sur des seuils critiques
 - Configurations dynamiques des seuils d'alerte

- **Notifications en temps réel :**
 - Push notifications via FCM

1.2.2 Portée et Périmètre

Le cadre du projet couvre l'ensemble de la chaîne de valeur des données :

Table 1.1: Domaines couverts par le projet

Phase	Description
Acquisition	OCR + validation des données
Traitement	Nettoyage, agrégation, analyse des tendances
Visualisation	Dashboard + historiques

1.3 Étude de l'existant

1.3.1 Présentation de l'étude de l'existant

Dans les hôpitaux et cliniques en Tunisie, il n'existe actuellement **aucun système de collecte de données automatisé** en salle de réveil. Le fonctionnement repose sur :

- **Un écran de surveillance centralisé** installé dans une salle administrative, permettant au personnel de suivre les signes vitaux de plusieurs patients à distance.
- **Une surveillance manuelle régulière** : Le personnel médical (infirmiers et médecins) doit se déplacer physiquement d'un patient à l'autre pour vérifier les constantes vitales, ce qui entraîne des **délais** dans la détection des situations critiques.

1.3.2 Critiques de l'existant

Manque de réactivité et d'optimisation des alertes

- Les soignants **ne reçoivent pas d'alertes sur leurs appareils mobiles**, ce qui oblige à une surveillance constante de l'écran centralisé.

Charge de travail accrue et inefficacité opérationnelle

- La **surveillance manuelle** et les déplacements constants du personnel entraînent une **perte de temps** et augmentent la charge de travail des soignants.
- **Risque d'erreurs humaines** dans la détection des anomalies, surtout en cas de forte affluence ou de fatigue du personnel.
- L'absence de **suivi automatisé des tendances des signes vitaux** empêche une anticipation des détériorations de l'état du patient.

1.4 Solution proposée

1.4.1 Collecte de données

- Solution basée sur caméra pour les moniteurs des signes vitaux, combinant **OpenCV** et **Tesseract OCR** pour extraire les données des écrans.

1.4.2 Gestion des alertes basée sur des règles

- Envoi des notifications via e-mails et notifications mobiles grâce au **Firestore Cloud Messaging (FCM)**.

1.5 Objectifs

- Collecte de données en temps réel dans les salles de réveil.
- Livraison des alertes en fonction de seuils définis.
- Amélioration des temps de réponse pour les situations critiques.
- Automatisation et réduction des interventions manuelles dans le traitement des données et alertes.

Chapter 2

Méthodologie adoptée

2.1 Choix de la méthodologie

2.1.1 Comparaison des méthodologies courantes

Le tableau suivant (Tableau 2.1) compare les méthodologies les plus couramment utilisées dans le développement logiciel, en mettant en évidence leurs caractéristiques clés :

Caractéristiques	Scrum	Waterfall	Lean	Kanban
Adaptabilité aux changements	✓	X	X	✓
Livraisons fréquentes	✓	X	X	X
Collaboration renforcée	✓	X	X	✓
Réponse rapide aux changements	✓	X	X	X
Gestion des risques	✓	✓	✓	✓
Niveau d'implication du client	Élevé	Faible	Moyen	Moyen
Efficacité des coûts	Élevée	Moyenne	Élevée	Moyenne
Assurance qualité	Continue	Finale	Continue	Continue
Scalabilité	Moyenne	Élevée	Élevée	Moyenne
Maintenance et support	Facile	Difficile	Facile	Facile

Table 2.1: Comparaison des méthodologies de développement logiciel

2.1.2 Rationale détaillé pour le choix de Scrum

Le choix de la méthodologie Scrum pour ce projet est motivé par plusieurs facteurs spécifiques au contexte du développement d'un système de collecte de données en temps réel et de gestion des alertes dans un environnement médical. Premièrement, l'approche itérative de Scrum permet de recueillir des retours continus des parties prenantes, notamment le personnel médical du SmartLab de la Faculté de Médecine de Monastir, pour affiner les exigences fonctionnelles et non fonctionnelles. Cela est particulièrement crucial dans un projet médical où la précision et la réactivité sont essentielles pour garantir la sécurité des patients.

Deuxièmement, Scrum offre une grande adaptabilité face aux incertitudes techniques, telles que l'extraction de données à partir de moniteurs via OCR. Ces sources de données présentent des défis distincts, notamment en termes de précision de l'OCR. Scrum permet de tester et valider ces composants dès les premiers sprints, réduisant ainsi les risques d'échec technique.

Enfin, la méthodologie Scrum favorise une collaboration étroite entre les développeurs, le Product Owner et le Scrum Master, assurant une communication fluide avec les parties prenantes médicales. Cette collaboration est essentielle pour répondre aux exigences changeantes du domaine médical, où les besoins peuvent évoluer rapidement en fonction des retours des utilisateurs ou des contraintes réglementaires [1,2].

2.2 Présentation de Scrum

Comme présenté dans la figure 2.1, Scrum est un framework agile qui repose sur des cycles de développement appelés sprints. Chaque sprint a une durée définie (généralement entre 2 et 4 semaines) et aboutit à

un produit fonctionnel et potentiellement livrable. Le processus Scrum comprend plusieurs éléments clés [3]:

- **Product Backlog** : Liste des fonctionnalités à développer, priorisée par le Product Owner en fonction des besoins des utilisateurs et des objectifs du projet.
- **Sprint Backlog** : Ensemble des tâches à réaliser pendant un sprint, défini par l'équipe de développement.
- **Sprint Planning** : Réunion permettant de définir les objectifs et les tâches du sprint.
- **Daily Scrum** : Brève réunion quotidienne de l'équipe pour suivre l'avancement et identifier les obstacles.
- **Sprint Review** : Présentation de l'incrément produit aux parties prenantes à la fin de chaque sprint.
- **Sprint Retrospective** : Analyse du sprint pour identifier les axes d'amélioration.



Figure 2.1: Processus Scrum.

2.2.1 Artefacts Scrum et leur application

Scrum repose sur trois artefacts principaux qui structurent le développement du projet : le *Product Backlog*, le *Sprint Backlog* et l'*Incrément*. Ces artefacts sont appliqués comme suit dans le cadre du projet [4]:

- **Product Backlog** : Le *Product Backlog* regroupe l'ensemble des fonctionnalités à développer, telles que définies dans le tableau 4.3 (voir section 4.3). Il est géré par le Product Owner, qui priorise les tâches en utilisant la méthode MoSCoW pour classer les fonctionnalités en *Must*, *Should*, *Could* et *Won't*. Ce backlog est régulièrement affiné pour intégrer les retours des parties prenantes, notamment le personnel médical, afin de garantir que les fonctionnalités prioritaires répondent aux besoins critiques du système.
- **Sprint Backlog** : À chaque sprint, un sous-ensemble de tâches est sélectionné à partir du *Product Backlog* pour constituer le *Sprint Backlog*. Par exemple, pour le Sprint 1, les tâches se concentrent sur l'intégration de la collecte de données basée sur caméra et l'extraction OCR (voir section ??). Ces tâches sont définies lors de la réunion de planification du sprint, en collaboration avec l'équipe de développement.

- **Incrément** : Chaque sprint produit un incrément fonctionnel et potentiellement livrable. Par exemple, à l'issue du Sprint 1, un pipeline OCR fonctionnel est livré, permettant la capture et l'extraction des données vitales à partir des moniteurs. Cet incrément peut être testé par le personnel médical pour valider sa précision et sa fiabilité avant d'intégrer de nouvelles fonctionnalités dans les sprints suivants.

2.2.2 Événements Scrum et leur implémentation

Scrum repose sur des événements clés qui structurent le processus de développement. Ces événements sont implémentés comme suit dans le cadre du projet [1]:

- **Planification de sprint** : Lors de la réunion de planification, l'équipe sélectionne les tâches du *Product Backlog* à inclure dans le *Sprint Backlog*, en estimant l'effort requis (voir tableau ?? pour le Sprint 1). Par exemple, pour le Sprint 1, l'intégration de l'OCR a été estimée à 5 jours, en tenant compte de la configuration des caméras et du pipeline de traitement d'images.
- **Daily Scrum** : Des réunions quotidiennes de 15 minutes sont organisées pour suivre l'avancement des tâches et identifier les obstacles. Par exemple, si des problèmes de qualité d'image affectent l'OCR, ils sont discutés et résolus rapidement, souvent en ajustant les paramètres de prétraitement d'OpenCV.
- **Revue de sprint** : À la fin de chaque sprint, l'incrément est présenté aux parties prenantes, comme le pipeline OCR fonctionnel à la fin du Sprint 1. Les médecins et infirmiers testent l'incrément pour valider sa précision et son utilité clinique, fournissant des retours pour le sprint suivant.
- **Rétrospective de sprint** : Après chaque sprint, l'équipe analyse les succès et les défis rencontrés. Par exemple, si des erreurs d'OCR sont détectées lors du Sprint 1, la rétrospective peut conduire à l'ajout de tests supplémentaires ou à l'amélioration du prétraitement des images dans le Sprint 2.

2.3 Les rôles Scrum

Dans Scrum, trois rôles principaux sont définies pour assurer le bon fonctionnement du processus de développement [5]:

- **Product Owner** : Représentant des parties prenantes, il définit et priorise les fonctionnalités à développer en fonction des besoins des utilisateurs et des objectifs du projet.
- **Scrum Master** : Responsable de la bonne application du framework Scrum. Il facilite la communication, aide à surmonter les obstacles et veille à l'amélioration continue des processus.
- **Équipe de développement** : Composée de développeurs, designers et autres spécialistes, elle est responsable de la mise en œuvre des fonctionnalités et de la livraison des incréments du produit.

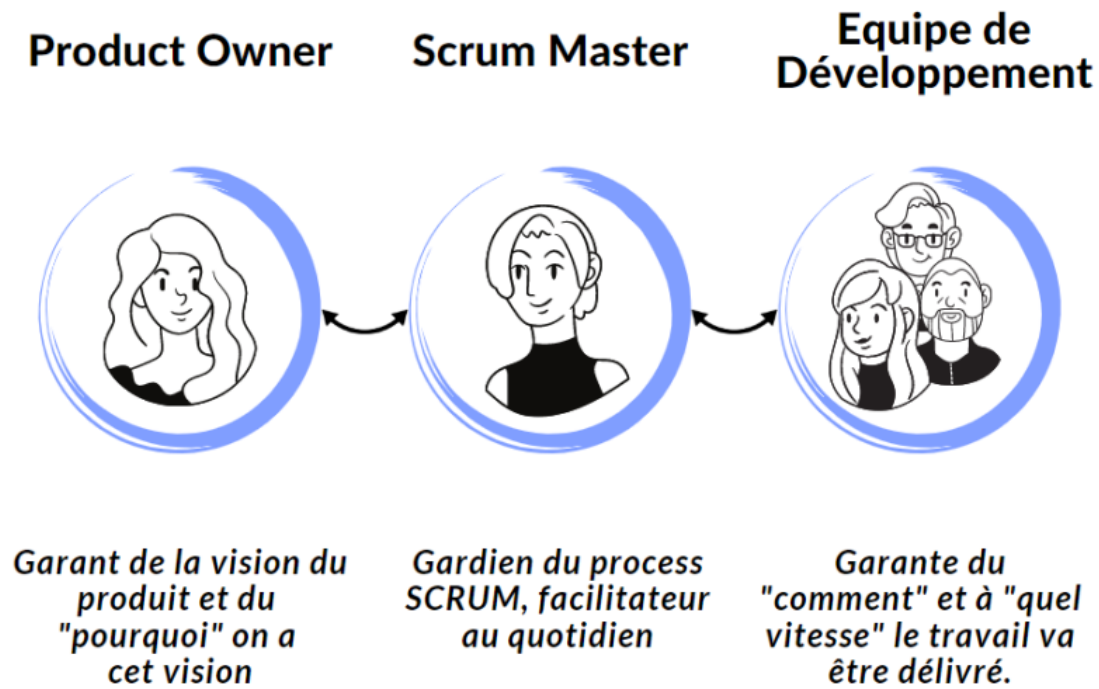


Figure 2.2: Rôles Scrum.

2.4 Gestion des risques avec Scrum

Le développement d'un système de collecte de données en temps réel et de gestion des alertes dans un contexte médical implique plusieurs risques, notamment techniques et opérationnels. Scrum offre un cadre efficace pour identifier, gérer et atténuer ces risques grâce à son approche itérative et ses mécanismes de revue réguliers [6].

- **Risques techniques** : L'extraction des données via OCR peut souffrir d'une précision insuffisante en raison de la qualité des images capturées ou de la variabilité des écrans des moniteurs. Ces risques sont atténués par des tests précoces dans les sprints initiaux, comme le Sprint 1, qui se concentre sur la validation de la collecte de données .
- **Risques opérationnels** : Les retards dans la livraison des alertes en temps réel peuvent compromettre la réactivité du personnel médical. Scrum permet de tester les performances des notifications dès le Sprint 2, en intégrant Firebase Cloud Messaging (FCM) et en validant les temps de réponse .
- **Revue et amélioration** : Les rétrospectives de sprint permettent d'identifier les problèmes liés aux processus, comme les goulots d'étranglement dans la communication entre l'équipe de développement et le personnel médical. Par exemple, si des retours indiquent des besoins supplémentaires pour l'interface utilisateur, ceux-ci peuvent être intégrés dans le *Product Backlog* pour le sprint suivant.

2.5 Engagement des parties prenantes dans Scrum

L'implication des parties prenantes est un pilier central de la méthodologie Scrum, particulièrement dans un projet médical où les besoins des utilisateurs finaux (médecins, infirmiers, administrateurs) sont critiques [1, 2]. Dans ce projet, les parties prenantes, notamment le personnel médical du SmartLab et les chercheurs de la Faculté de Médecine de Monastir, jouent un rôle actif à plusieurs niveaux :

- **Revue de sprint** : À la fin de chaque sprint, les parties prenantes participent aux réunions de revue pour évaluer les incréments livrés, tels que le pipeline OCR du Sprint 1 ou le système de

notification du Sprint 2. Leurs retours permettent de valider la conformité des fonctionnalités aux besoins cliniques, par exemple en vérifiant la précision des données extraites ou l'ergonomie de l'interface utilisateur.

- **Affinement du Product Backlog :** Le Product Owner collabore avec les parties prenantes pour prioriser les fonctionnalités, en utilisant la méthode MoSCoW pour classer les tâches (voir tableau 4.3) [3]. Par exemple, les médecins peuvent insister sur la priorisation des alertes critiques (M-02) pour garantir une réactivité maximale.
- **Conformité réglementaire :** Les parties prenantes, en particulier les experts médicaux, contribuent à garantir que le système respecte les réglementations médicales, comme la protection des données des patients via le chiffrement (M-07). Leur expertise est essentielle pour aligner le développement sur les normes éthiques et légales.

2.6 Outils de support à Scrum

Pour faciliter la mise en œuvre de Scrum, plusieurs outils numériques sont utilisés pour gérer les processus, la communication et le suivi des tâches :

- **ClickUp :** Utilisé pour gérer le *Product Backlog* et le *Sprint Backlog*, ClickUp permet de suivre les tâches, d'assigner des responsabilités et de visualiser l'avancement des sprints à l'aide de tableaux Kanban ou Scrum [7]. Par exemple, les tâches du Sprint 1, comme l'intégration de l'OCR (M-02), sont suivies via des tickets ClickUp avec des estimations de temps et des statuts mis à jour quotidiennement.
- **Slack :** Cette plateforme de communication permet des échanges en temps réel entre l'équipe de développement, le Product Owner et le Scrum Master [8]. Les *Daily Scrum* peuvent être partiellement gérés via des canaux dédiés, où les membres signalent leurs progrès et obstacles.
- **Git :** Utilisé pour le contrôle de version, Git permet de gérer le code source du projet et de faciliter la collaboration entre les développeurs [9]. Les commits sont alignés avec les tâches du *Sprint Backlog*, garantissant une traçabilité des contributions.

2.7 Représentation visuelle du processus Scrum

La figure 2.3 illustre le processus Scrum adapté au projet, montrant la séquence des événements et les boucles de rétroaction. Chaque sprint commence par une planification, suivie de réunions quotidiennes (*Daily Scrum*), d'une revue de sprint pour présenter l'incrément, et d'une rétrospective pour améliorer le processus. Ce cycle itératif garantit que les fonctionnalités, comme la collecte de données via OCR ou la gestion des alertes, sont développées et validées progressivement.

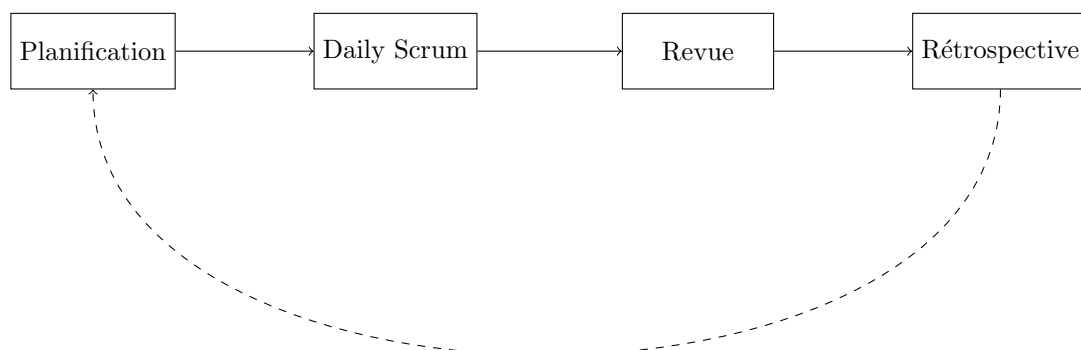


Figure 2.3: Processus Scrum adapté au projet.

Chapter 3

Étude architecturale du projet

3.1 Introduction

Ce chapitre présente l'architecture globale du système de collecte de données en temps réel et de gestion des alertes pour les machines en salle de réveil. L'objectif est de décrire les choix technologiques, les composants clés et les interactions entre les différents modules pour répondre aux besoins fonctionnels et non fonctionnels identifiés.

Ce système est conçu pour répondre à un contexte hospitalier sensible, exigeant une haute disponibilité, une sécurité accrue des données, et une capacité d'adaptation à différents environnements techniques.

3.2 Architecture globale

Le système repose sur une architecture modulaire et évolutive, organisée en trois couches principales :

- **Couche de collecte des données :**
 - Intègre des *Moniteurs* médicaux avec *Caméras IP* pour capturer les frames des écrans.
- **Couche de traitement et stockage :**
 - Utilise OpenCV pour le prétraitement des images, *YOLO* pour détecter les zones de données (e.g., fréquence cardiaque, SpO2) dans les images capturées, et Tesseract OCR pour l'extraction des valeurs vitales.
 - *Flask* pour le service de traitement d'images, qui communique avec le *Backend* Laravel.
 - Un *Backend* Laravel pour la gestion des utilisateurs, des données et des alertes.
 - Une base de données MySQL pour le stockage des données des patients, des mesures et des historiques d'alertes.
- **Couche de visualisation et notification :**
 - Interfaces *Application Web* (React) et *Application Mobile* (Flutter) pour la consultation des données et la gestion des alertes, connectées via REST API.
 - Notifications en temps réel via Firebase Cloud Messaging (FCM) pour les alertes push, et envoi d'e-mails pour les alertes critiques.

La figure 3.1 illustre cette organisation en couches.

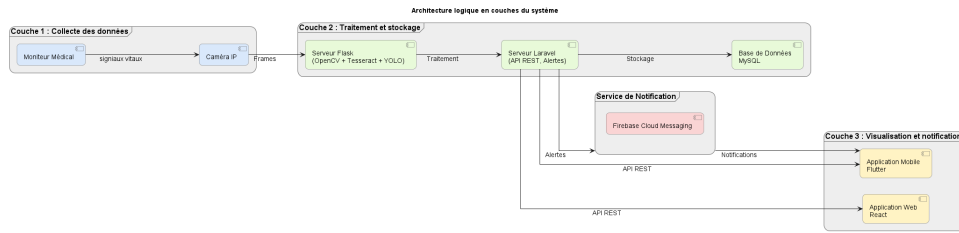


Figure 3.1: Architecture globale du système

3.3 Diagrammes d'architecture

3.3.1 Diagramme de déploiement

Le système repose sur une architecture distribuée intégrant des dispositifs médicaux, des serveurs applicatifs, des services de traitement d'image, et des interfaces utilisateur. Voici les principaux composants matériels et logiciels déployés, regroupés par couche :

Couche de collecte des données :

- **Moniteurs médicaux** : collecte des données vitales.
- **Caméras IP** : capture des écrans des moniteurs.

Couche de traitement et stockage :

- **Serveur backend** : héberge l'API Laravel, le service Flask de traitement d'images, et le modèle YOLO pour la détection des zones de données.
- **Base de données MySQL** : stockage des données patient, mesures et alertes.
- **Services de traitement d'images** : YOLO pour la détection des régions d'intérêt, OpenCV pour le prétraitement, et Tesseract pour l'extraction OCR, exécutés sur le serveur Flask.

Couche de visualisation et notification :

- **Appareils mobiles** : consultation en temps réel des alertes via l'application Flutter.
- **Application web React** : interface utilisateur pour le personnel médical.
- **Services de notification** : Firebase Cloud Messaging pour les alertes push.

Infrastructure et support :

- **Réseau sécurisé** : assure la communication sécurisée entre tous les composants.

Connexions et protocoles :

- Communication via **REST API** (HTTP/HTTPS) et **WebSockets**, sécurisée par des mécanismes d'authentification (Sanctum) et d'autorisation.
- Flux vidéo des caméras traité localement ou sur le cloud, avec analyse par YOLO et OCR.
- Notifications push envoyées via **Firebase Cloud Messaging (FCM)**.

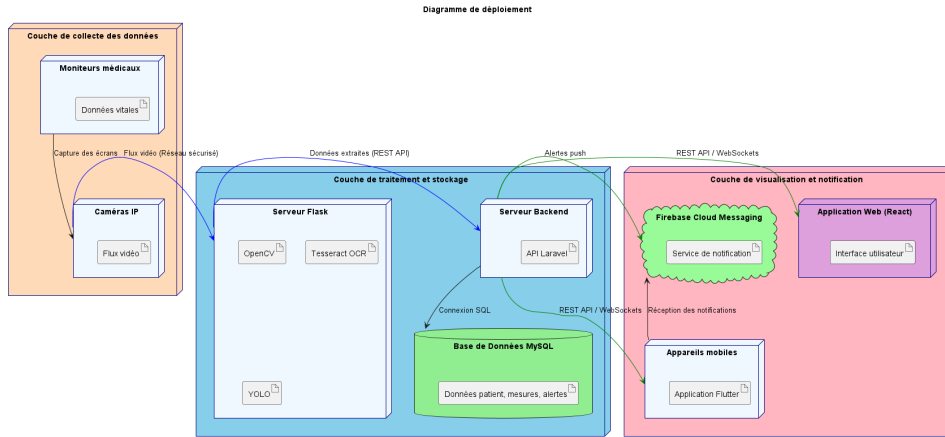


Figure 3.2: Diagramme de déploiement du système, incluant YOLO pour la détection des zones de données.

3.3.2 Diagramme de composants

Backend :

- Modules Python pour la détection des zones de données (YOLO), l'OCR (Tesseract), et l'analyse des données.
- Services Laravel pour l'authentification, la gestion des alertes et les notifications.

Frontend :

- Tableaux de bord React pour la visualisation des données.
- Application Flutter pour les alertes mobiles.

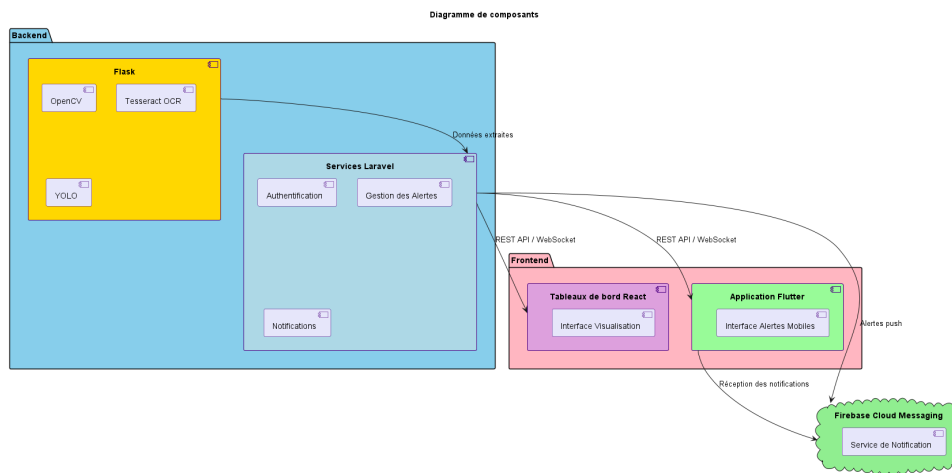


Figure 3.3: Diagramme de composants du système, incluant le module YOLO.

3.4 Justification des choix technologiques

Cette section détaille les raisons ayant conduit au choix des technologies clés, en conciliant performance, efficacité économique, évolutivité et intégration avec les systèmes existants.

• **YOLO (You Only Look Once) :**

- Utilisé pour détecter les zones de données (régions d'intérêt) sur les écrans des moniteurs médicaux, permettant une extraction précise des valeurs vitales par OCR [10].

- Offre une détection rapide et robuste des bounding boxes autour des paramètres vitaux (e.g., fréquence cardiaque, SpO2), même dans des conditions d'éclairage variables.
- Complète OpenCV pour le prétraitement des images et Tesseract pour l'extraction de texte, réduisant les erreurs d'OCR en isolant les zones pertinentes.
- **Tesseract OCR :**
 - Offre une solution efficace pour extraire des données des zones identifiées par YOLO, avec une grande précision pour les données numériques [11].
 - Constitue une alternative économique à la modernisation des moniteurs de signes vitaux, réduisant ainsi les coûts matériels [12].
- **Laravel et Python :**
 - Laravel fournit un framework robuste basé sur l'architecture MVC, prenant en charge l'authentification sécurisée des utilisateurs, le développement d'API et l'évolutivité [13].
 - Python améliore le traitement d'images grâce à des bibliothèques comme OpenCV et PyTorch (pour YOLO), offrant une flexibilité pour l'analyse en temps réel [14].
- **Firebase Cloud Messaging (FCM) :**
 - Garantit des notifications push rapides et multiplateformes (Android/iOS) avec une latence minimale, optimisées pour les alertes médicales [15].

3.5 Flux de données

Cette section décrit le flux de données à travers le système, organisé en trois étapes clés : collecte, traitement et notification.

- **Collecte :**
 - Les caméras capturent les données affichées sur les écrans des appareils médicaux. Le modèle YOLO détecte les zones de données (e.g., fréquence cardiaque, pression artérielle) dans les images, qui sont ensuite prétraitées avec OpenCV et analysées par Tesseract OCR pour extraire les données des signes vitaux.
- **Traitement :**
 - Le backend traite les données extraites, les compare à des seuils critiques prédéfinis et génère des alertes lorsque ces seuils sont dépassés.
- **Notification :**
 - Les alertes sont distribuées via Firebase Cloud Messaging (FCM) pour les notifications push, par e-mail, SMS, et affichées sur les interfaces web et mobiles en temps réel.

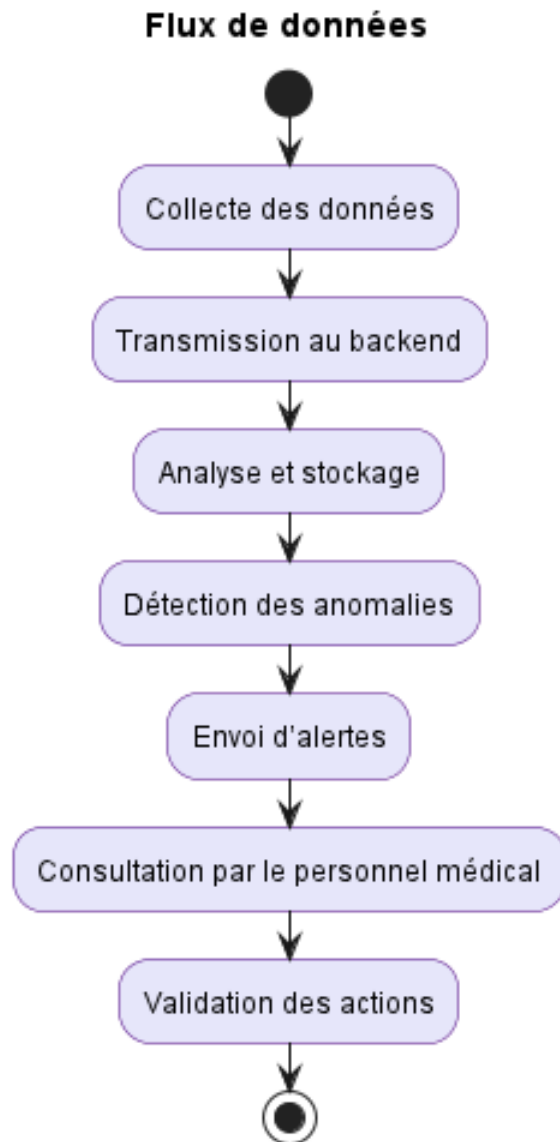


Figure 3.4: Schéma général du flux de données, incluant la détection des zones par YOLO.

3.6 Architecture physique et fonctionnement

3.6.1 Composants matériels

- **Moniteurs** : Collectent les données vitales (fréquence cardiaque, pression artérielle, saturation en oxygène, etc.) et les affichent sur leurs écrans.
- **Caméras** : Capturent les écrans des moniteurs et transmettent les images au service de traitement pour analyse par YOLO et OCR.
- **Serveurs** : Stockent et traitent les données, hébergés localement, exécutant YOLO, OpenCV, et Tesseract pour l'analyse d'images.
- **Réseau de communication** : Assure la transmission sécurisée des données entre les moniteurs, caméras, serveurs et applications.
- **Appareils mobiles** : Permettent au personnel médical de recevoir des notifications push et d'accéder aux données en temps réel.

3.6.2 Fonctionnement

Le système fonctionne en trois étapes principales :

1. Collecte des données :

- Les écrans des moniteurs sont capturés par des caméras. Le modèle YOLO identifie les zones de données (e.g., fréquence cardiaque, SpO2), suivies par un prétraitement avec OpenCV et une extraction des valeurs vitales via Tesseract OCR.

2. Traitement des données :

- Les données sont analysées en temps réel pour détecter des anomalies par rapport aux seuils prédéfinis.
- Les données sont stockées dans une base MySQL pour une analyse ultérieure.

3. Génération des alertes :

- Les alertes sont générées en cas de dépassement des seuils.
- Elles sont envoyées via FCM, e-mails ou SMS, et affichées sur les interfaces web/mobile.

3.6.3 Spécifications logicielles

• Backend :

- *Laravel* : Gestion des utilisateurs, des données et des services de notification (FCM).
- *Python* : Traitement des données, intégration de YOLO pour la détection des zones de données, OCR (OpenCV, Tesseract), et analyse d'images.

• Frontend :

- *React* : Interface web pour la visualisation interactive des données et la gestion des alertes.
- *Flutter* : Application mobile pour les notifications et l'accès aux données en temps réel.

• Base de données : MySQL pour le stockage des données des patients, des alertes et des historiques.

• Protocoles de communication :

- REST API pour les échanges entre frontend, backend et application mobile.
- WebSockets pour les notifications en temps réel.

• Outils complémentaires :

- YOLOv8 pour la détection des zones de données, intégré avec PyTorch.
- Docker pour la conteneurisation.
- Git pour le contrôle de version.
- Swagger pour tester les API REST.

3.6.4 Flux de communication

Le flux commence par la collecte des données à partir des moniteurs (via YOLO pour la détection des zones de données et OCR pour l'extraction). Les données sont transmises au backend via un réseau sécurisé. Le backend analyse les données en temps réel, les stocke dans MySQL, et génère des alertes en cas d'anomalies. Ces alertes sont envoyées au personnel médical via FCM. Les utilisateurs peuvent consulter les données et alertes via l'application web ou mobile.

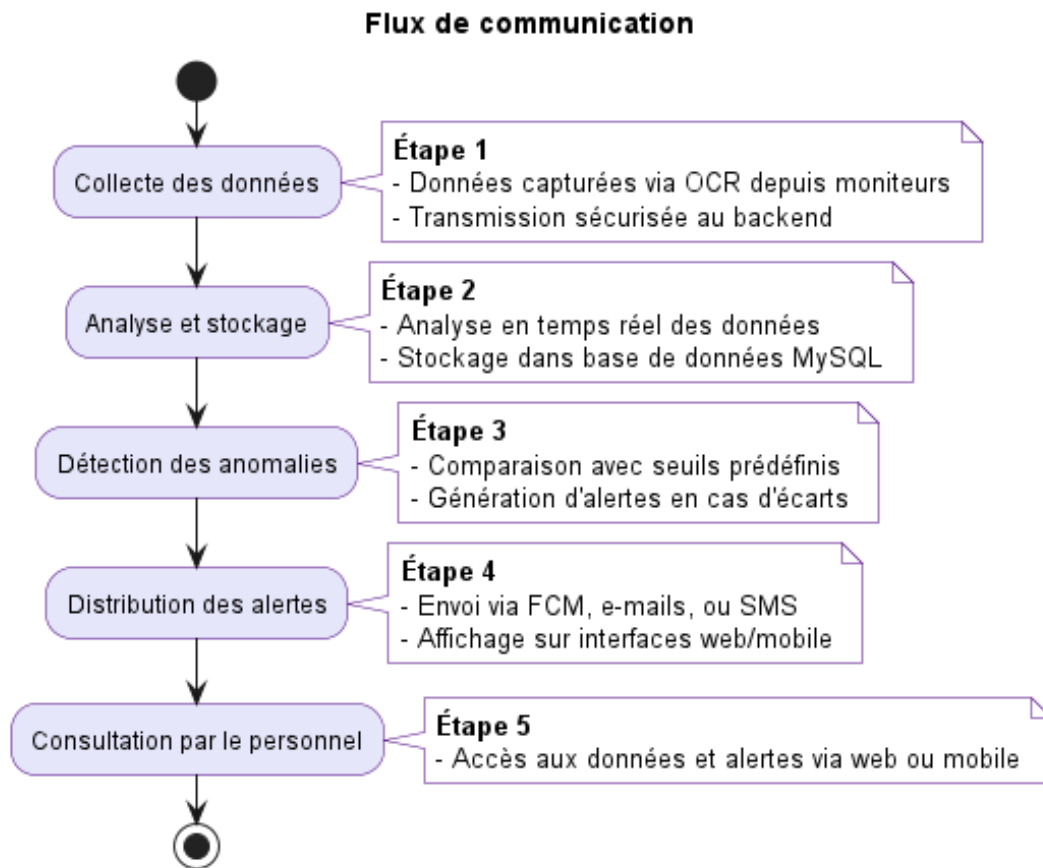


Figure 3.5: Schéma général du flux de communication, incluant YOLO pour la détection des zones.

3.7 Justification de l'architecture

L'architecture proposée garantit :

- **Modularité** : Séparation claire des couches (collecte avec YOLO, traitement, notification) pour une maintenance aisée.
- **Réactivité** : Alertes en temps réel pour une réponse rapide aux situations critiques, soutenues par une détection précise via YOLO.
- **Interopérabilité** : Compatibilité avec les technologies YOLO, OCR, et les standards d'échange de données médicales.
- **Flexibilité** : Déploiement possible sur serveurs locaux ou cloud, adaptable aux besoins hospitaliers.

3.8 Conclusion

L'architecture proposée, enrichie par l'intégration de YOLO pour une détection précise des zones de données, assure une collecte fiable, un traitement efficace et une diffusion rapide des alertes, tout en respectant les contraintes de sécurité et d'interopérabilité. Sa modularité permet des évolutions futures, comme l'intégration de nouveaux appareils ou l'ajout de fonctionnalités analytiques.

Chapter 4

Sprint 0

4.1 Introduction

Ce chapitre présente le **Sprint 0**, qui marque la phase initiale de la réalisation de notre projet. Le **Sprint 0** est une étape cruciale où nous posons les fondations du projet en identifiant les acteurs clés, en capturant les besoins fonctionnels et non fonctionnels, et en définissant la méthodologie de travail qui guidera le développement. Cette phase permet de clarifier les objectifs, les attentes et les contraintes du projet, tout en établissant un cadre de travail structuré pour les étapes ultérieures.

4.2 Capture des besoins

4.2.1 Identification des acteurs

Dans un premier temps, nous identifions les acteurs du système, c'est-à-dire les entités externes qui interagiront avec l'application. Ces acteurs jouent un rôle central dans la définition des fonctionnalités et des services que le système doit fournir.

Acteur	Description
Médecin	Consulte les données des patients, reçoit les alertes critiques et valide les actions nécessaires.
Infirmier(ère)	Surveille les patients, reçoit les alertes en fonction de ses horaires et prend des mesures immédiates.
Administrateur du Système	Gère les utilisateurs, configure les seuils d'alerte et assure le bon fonctionnement du système.
Moniteur de Signes Vitaux	Source de données : envoie des informations vitales via API ou caméra.
Caméra	Capturent les écrans des machines et les transmet pour analyse.

Table 4.1: Acteurs du système et leurs rôles

4.2.2 Besoins fonctionnels et non fonctionnels

Cette section détaille les besoins fonctionnels et non fonctionnels du système, qui reflètent les exigences des utilisateurs ainsi que les contraintes techniques à respecter.

Besoins Fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités attendues du système :

- **Collecte des données** : Le système doit être capable de collecter des données en temps réel à partir des moniteurs de signes vitaux et des caméras.
- **Détection et reconnaissance des valeurs vitales** : Utilisation de l'OCR pour extraire les valeurs vitales à partir des images capturées.
- **Génération et gestion des alertes** : Le système doit déclencher des alertes en fonction des seuils définis et les acheminer au personnel médical concerné.
- **Affichage des données** : Une interface web et mobile permettant aux médecins et infirmiers d'accéder aux données patient.
- **Authentification et gestion des utilisateurs** : Gestion des rôles et permissions des utilisateurs (administrateurs, médecins, infirmiers).
- **Notifications** : Envoi d'alertes par e-mail, SMS ou notifications push via Firebase Cloud Messaging (FCM).
- **Stockage et historique des données** : Enregistrement des valeurs mesurées et des alertes pour analyse et suivi.

Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les contraintes techniques et de performance du système :

- **Disponibilité** : Le système doit être accessible 24/7 avec une tolérance aux pannes minimale.
- **Sécurité** : Protection des données médicales via chiffrement et authentification sécurisée.
- **Interopérabilité** : Compatibilité avec différents modèles de moniteurs (Mindray, Edan).
- **Scalabilité** : Capacité à gérer une augmentation du nombre de patients et d'appareils connectés.
- **Temps de réponse** : Le traitement des alertes doit être instantané pour garantir la réactivité du personnel médical.
- **Conformité** : Respect des réglementations médicales.

4.2.3 Diagramme des cas d'utilisation global

Montrez les prototypes initiaux pour les interfaces du système (par exemple, visualisation des données, gestion des alertes).

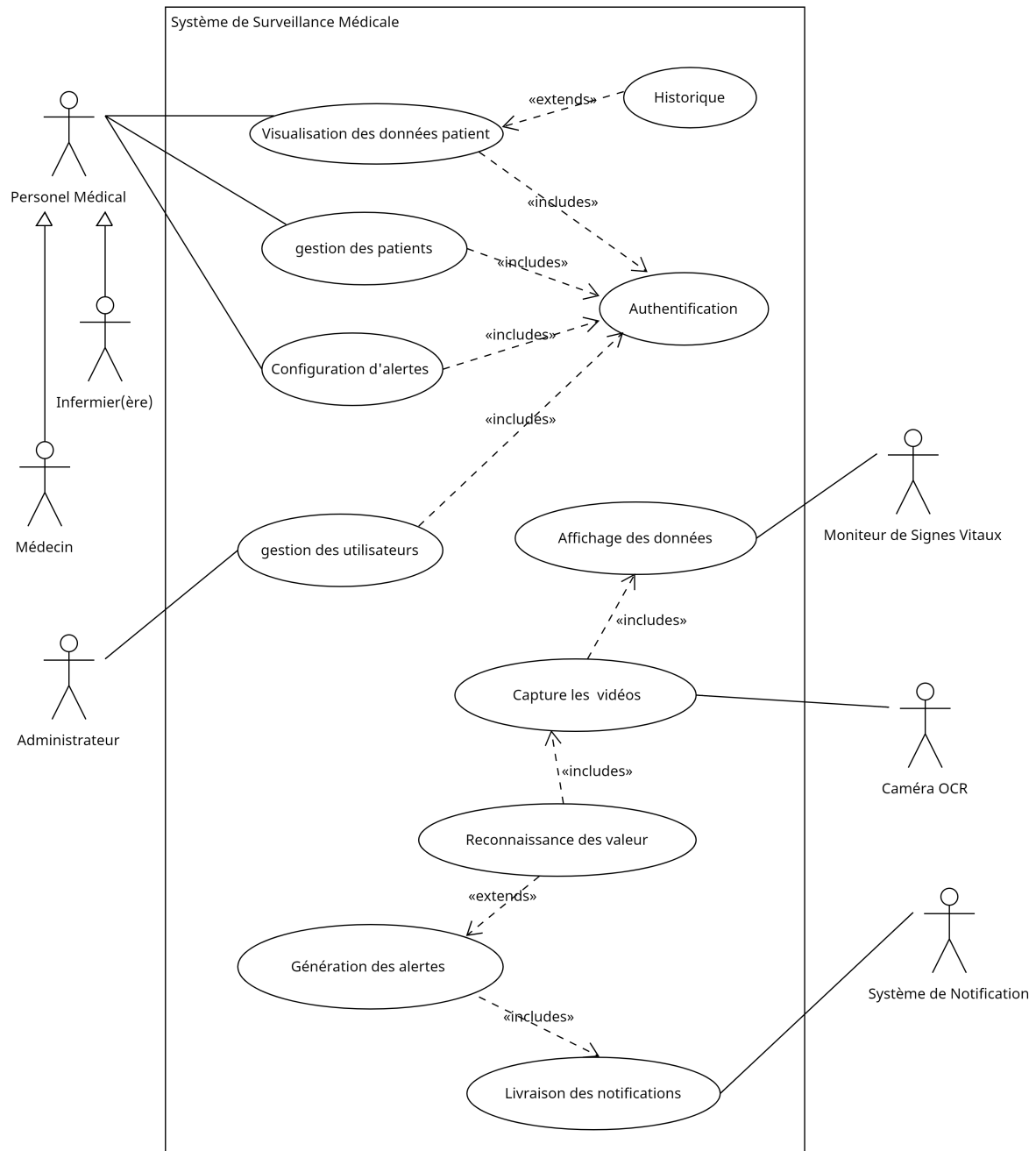


Figure 4.1: Diagramme des cas d'utilisation global

4.3 Pilotage du projet avec Scrum

Le projet sera géré selon la méthodologie **Scrum** 2.2, favorisant une approche agile avec des cycles itératifs appelés **sprints**. Le **Product Owner**, l'équipe de développement et le **Scrum Master** collaboreront pour assurer une livraison progressive des fonctionnalités clés.

Acteurs Scrum et Missions

Rôle	Mission	Acteur
Product Owner	- Définir et prioriser les fonctionnalités du produit (Product Backlog). - Assurer que l'équipe travaille sur les fonctionnalités les plus importantes. - Valider les livrables à la fin de chaque sprint.	Pr Ammeri Charfeddine
Équipe de Développement	- Concevoir, développer et tester les fonctionnalités du produit. - Participer aux réunions de planification et de revue de sprint. - Assurer la qualité du code et des livrables.	Zaibi Racha
Scrum Master	- Faciliter les réunions Scrum (Daily Stand-up, Sprint Planning, Sprint Review, Retrospective). - Supprimer les obstacles qui empêchent l'équipe de progresser. - Assurer que l'équipe suit les principes Scrum.	Ouni Ghada

Table 4.2: Acteurs du projet

Organisation du Backlog

Le **Product Backlog** géré via ClickUp 2.6 regroupe l'ensemble des fonctionnalités à développer pour le système. Chacune est formulée sous forme de *User Story*, garantissant une approche centrée sur l'utilisateur.

Chaque fonctionnalité sera développée et livrée en plusieurs **sprints**, selon les priorités établies par le *Product Owner*. Elles sont classées selon la méthode **MoSCoW**, qui définit les niveaux de priorité pour chaque tâche.

Backlog du Projet

ID	Tâche	Priority
M-01	Détection et reconnaissance des valeurs vitales via OCR pour les moniteurs des signes vitaux.	M
M-02	Génération et gestion des alertes en fonction des seuils prédéfinis.	M
M-03	Envoi d'alertes aux médecins et infirmiers via notifications push (FCM) et e-mail.	M
M-04	Interface web et mobile pour visualiser les données des patients et gérer les alertes.	M
M-05	Authentification et gestion des utilisateurs (médecins, infirmiers, administrateurs).	M
M-06	Stockage et historique des données.	M
M-07	Conformité aux réglementations médicales (sécurité des données).	M
S-02	Configuration des seuils d'alerte personnalisés par patient.	S
S-03	Gestion des horaires de filtrage des alertes pour le personnel médical.	S
S-04	Interface d'administration pour configurer les seuils d'alerte et gérer les utilisateurs.	S
S-05	Notifications push.	S
C-04	Support multilingue pour l'interface utilisateur.	C
C-05	Gestion des alertes par priorité (critique, moyen, faible).	C
C-06	Historique des alertes avec possibilité de filtrage par date, patient, ou type d'alerte.	C
W-03	Analyse avancée des données avec intelligence artificielle pour le diagnostic médical.	W

ID	Tâche	Priority
----	-------	----------

Table 4.3: Backlog du Projet avec Priorités MoSCoW

Clé des Priorités MoSCoW

- **M (Must)** : Tâches critiques nécessaires au bon fonctionnement du système.
- **S (Should)** : Tâches importantes qui apportent une valeur significative mais ne sont pas essentielles pour la première version.
- **C (Could)** : Fonctionnalités optionnelles pouvant être ajoutées si le temps et les ressources le permettent.
- **W (Won't Have)** : Tâches hors du périmètre de la phase actuelle du projet.

4.3.1 Planification des sprints

Une fois le backlog du produit finalisé, nous avons organisé la réunion de planification de sprint. L'objectif de cette réunion était de construire le backlog de sprint en se basant sur le backlog de produit défini par le Product Owner. À l'issue de cette réunion, nous avons identifié les durées prévisionnelles du travail à effectuer pour chaque sprint. Pour ce projet, nous avons divisé le travail en trois sprints, chacun d'une durée de 3 semaines, regroupés en une seule release. La figure 2.3 illustre la répartition des sprints pour notre système.

Plan des sprints :

- **Sprint 1 : Collecte de données basée sur caméra**
 - **Objectif** : Mettre en place un système de collecte de données à partir des moniteurs des signes vitaux en utilisant des caméras et des techniques de reconnaissance optique de caractères (OCR).
 - **Deliverables** :
 - * Intégration de la caméra IP avec le système.
 - * Pipeline OCR pour extraire les valeurs vitales (fréquence cardiaque, pression artérielle, SpO2).
 - **Durée** : 3 semaines.
 - **Tâches clés** :
 - * Configuration des caméras IP (montage fixe, résolution 1080p).
 - * Installation et configuration des logiciels OCR (OpenCV, Tesseract).
 - * Développement du pipeline de prétraitement et extraction d'images.
 - **Critères d'acceptation** :
 - * Le pipeline OCR extrait les valeurs vitales avec une précision d'au moins 95%, validée par comparaison avec 10 lectures manuelles.
 - * Les images sont traitées avec une latence inférieure à 1 seconde par frame dans 100% des tests.
 - * Le pipeline est compatible avec les moniteurs Mindray et Edan, testé sur au moins 5 scénarios différents.
 - **Risques** :
 - * Mauvaise qualité d'image affectant la précision de l'OCR.
 - * Solution : Utilisation de caméras haute résolution et optimisation du prétraitement des images avec OpenCV.
- **Sprint 2 : Validation des données et gestion des alertes**
 - **Objectif** : Valider les données collectées et développer un système de gestion des alertes basé sur des seuils prédéfinis.

- **Deliverables :**
 - * Validation des données OCR par rapport aux données réelles.
 - * Module de gestion des alertes avec seuils prédéfinis.
 - * Notifications push via Firebase Cloud Messaging (FCM).
- **Durée :** 3 semaines.
- **Tâches clés :**
 - * Tests de validation des données extraites (comparaison avec lectures manuelles).
 - * Développement du module de détection des anomalies basé sur des seuils.
 - * Intégration de FCM pour les notifications push sur mobile.
- **Critères d'acceptation :**
 - * Les données extraites correspondent aux valeurs réelles avec une précision de 98% dans 20 tests.
 - * Les alertes sont déclenchées en moins de 2 secondes lorsque les seuils sont dépassés (e.g., fréquence cardiaque supérieure à 100 bpm ou inférieure à 60 bpm, pression artérielle systolique supérieure à 180 mmHg ou inférieure à 90 mmHg, SpO2 inférieure à 90%).
 - * Les notifications push sont reçues avec un taux de succès de 99% sur Android et iOS dans un environnement de test.
- **Dépendances :** Données collectées et pipeline OCR fonctionnel du Sprint 1.
- **Risques :**
 - * Retards dans la validation des données en raison d'erreurs OCR.
 - * Solution : Ajout de tests unitaires et manuels supplémentaires pour l'OCR.
- **Sprint 3 : Interface utilisateur et stockage des données**
 - **Objectif :** Développer une interface utilisateur pour visualiser les données et alertes, et mettre en place le stockage des données.
 - **Deliverables :**
 - * Tableau de bord web (React) pour afficher les données vitales et alertes.
 - * Application mobile (Flutter) pour les notifications et la consultation.
 - * Base de données MySQL pour stocker les données et historiques.
 - **Durée :** 3 semaines.
 - **Tâches clés :**
 - * Conception et développement du tableau de bord web avec React.
 - * Développement de l'application mobile Flutter avec intégration FCM.
 - * Configuration de la base de données MySQL pour le stockage des données et alertes.
 - **Critères d'acceptation :**
 - * Le tableau de bord affiche les données vitales et alertes en temps réel avec un rafraîchissement toutes les 5 secondes, validé par 5 utilisateurs médicaux.
 - * L'application mobile permet la consultation des alertes avec une ergonomie validée par 3 infirmiers (aucun problème d'utilisabilité signalé).
 - * Les données sont stockées dans MySQL avec 100% d'intégrité (aucune perte de données dans 50 tests).
 - **Dépendances :** Données validées et système d'alertes du Sprint 2.
 - **Risques :**
 - * Complexité dans l'intégration des interfaces web et mobile.
 - * Solution : Prototypage préliminaire et tests d'utilisabilité itératifs.

4.4 Environnement de travail

4.4.1 Environnement matériel

Pour assurer le bon fonctionnement du système, les exigences matérielles suivantes doivent être respectées:

- **Caméras pour les moniteurs des signes vitaux :**
 - **Type :** Caméras IP.
 - **Rôle :** Capture des écrans des moniteurs des signes vitaux pour l'extraction des données via OCR (Reconnaissance Optique de Caractères).
 - **Configuration :** Montage fixe sur les moniteurs pour une capture stable et précise.
- **Serveurs pour le stockage et le traitement des données :**
 - **Type :** Serveurs dédiés (serveurs locaux).
 - **Rôle :** Stockage des données collectées, traitement en temps réel, et génération des alertes.
 - **Spécifications techniques :**
 - * **Processeur :** Multi-core (i5).
 - * **Mémoire RAM :** 16 Go.
 - * **Stockage :** Disque SSD de 500 Go pour un accès rapide aux données.
 - * **Connectivité :** Réseau Ethernet pour une transmission rapide des données.
- **Réseau de communication :**
 - **Type :** Réseau local (LAN) sécurisé avec accès à Internet.
 - **Rôle :** Assurer la communication entre les caméras, les serveurs et les applications mobiles/web.
- **Appareils mobiles pour le personnel médical :**
 - **Type :** Smartphones ou tablettes.
 - **Rôle :** Recevoir des notifications push en temps réel et accéder à l'interface mobile du système.
 - **Exigences :**
 - * **Version minimale :** Android 8.0 (Oreo) ou iOS 12.
 - * **Connexion :** Wi-Fi ou 4G/5G pour une réception rapide des alertes.

4.4.2 Environnement de développement

Pour le développement du système, les outils suivants seront utilisés :

- **Langages de programmation :**
 - **Python :**
 - * Version : Python 3.11.0 .
 - * Rôle : Utilisé pour le traitement des données, la collecte à partir des images via OCR, et l'intégration des bibliothèques de vision par ordinateur (OpenCV, Tesseract OCR).
 - * Frameworks : Flask ou Django pour le développement backend.
 - **Laravel :**
 - * Version : Laravel 11.44.2 .
 - * Rôle : Utilisé pour le développement du backend, la gestion des utilisateurs, et l'intégration des services de notification (e-mail, FCM).
 - * Fonctionnalités clés : ORM (Eloquent), gestion des routes, et système de templates (Blade).
 - **React :**
 - * Version : React 18.3.1 .

- * Rôle : Utilisé pour le développement de l'interface web, permettant une visualisation interactive des données des patients et la gestion des alertes.
- * Bibliothèques associées : Redux pour la gestion de l'état, Axios pour les requêtes HTTP.
- **Flutter** :
 - * Version : Flutter 3.27.0 .
 - * Rôle : Utilisé pour le développement de l'application mobile, permettant au personnel médical de recevoir des notifications et d'accéder aux données des patients en temps réel.
 - * Fonctionnalités clés : Interface native pour Android et iOS, intégration avec Firebase pour les notifications push.
- **Bibliothèques et outils** :
 - **OpenCV** :
 - * Rôle : Utilisé pour le traitement d'images et la détection des régions d'intérêt (ROI) dans les moniteurs de signes vitaux.
 - * Fonctionnalités clés : Manipulation d'images, détection de texte, et prétraitement des données pour l'OCR.
 - **Tesseract OCR** :
 - * Rôle : Utilisé pour la reconnaissance optique de caractères (OCR) afin d'extraire les valeurs vitales des images capturées par les caméras.
 - * Fonctionnalités clés : Support multilingue, haute précision pour les chiffres et les textes médicaux.
 - **Firebase Cloud Messaging (FCM)** :
 - * Rôle : Utilisé pour envoyer des notifications push en temps réel aux appareils mobiles du personnel médical.
 - * Fonctionnalités clés : Intégration avec Flutter et Laravel, support pour Android et iOS.
 - **Docker** :
 - * Rôle : Utilisé pour la conteneurisation des applications, permettant un déploiement facile et une gestion des dépendances.
 - * Fonctionnalités clés : Isolation des environnements, déploiement sur différents serveurs.
 - **Git** :
 - * Rôle : Utilisé pour le contrôle de version et la collaboration entre les membres de l'équipe de développement.
 - * Plateforme : GitHub ou GitLab pour le stockage des dépôts et la gestion des branches.

4.4.3 Environnement logiciel

Pour assurer le bon fonctionnement du système, les exigences logicielles suivantes doivent être respectées :

- **Systèmes d'exploitation** :
 - **Postes de développement** :
 - * Windows 11.
 - * Rôle : Environnement de développement (backend & frontend & mobile).
 - * Configuration : 16 Go de RAM, processeur Intel i5.
 - **Appareils mobiles** :
 - * Android : Version 14.
 - * Rôle : Exécution de l'application mobile pour le personnel médical.
- **Systèmes de base de données** :
 - **Base de données principale** :
 - * MySQL : Version 9.1.0 .

- * Rôle : Stockage des données des patients, des alertes, et des historiques.
- * Configuration : Base de données relationnelle avec support des transactions ACID.
- **Protocoles de communication :**
 - **API REST :**
 - * Rôle : Interface de communication entre les différents composants du système (frontend, backend, mobile).
 - * Sécurité : Utilisation de tokens (Bearer Tokens) pour l'authentification et l'autorisation.
 - * Documentation : Swagger/OpenAPI pour une documentation interactive des API.
 - **WebSockets :**
 - * Rôle : Communication en temps réel entre le backend et les applications mobiles/web pour les notifications push.
 - * Utilisation : Envoi instantané des alertes critiques au personnel médical.
- **Autres logiciels et outils :**
 - **IDE (Environnements de développement intégrés) :**
 - * Visual Studio Code : Pour le développement en Python, React, et Laravel.
 - * Android Studio : Pour le développement de l'application mobile Flutter.
 - **Gestion des dépendances :**
 - * Composer : Pour la gestion des packages PHP (Laravel).
 - * npm : Pour la gestion des packages JavaScript (React).
 - * pip : Pour la gestion des packages Python.
 - **Outils de tests :**
 - * Postman & Swagger : Pour tester les API.

Bibliography

- [1] Ken Schwaber and Jeff Sutherland. *Scrum: The art of doing twice the work in half the time*. Crown Business, 2004.
- [2] Kenneth S. Rubin. *Essential Scrum: A practical guide to the most popular Agile process*. Addison-Wesley, 2012.
- [3] Atlassian. What is scrum? <https://www.atlassian.com/agile/scrum>, 2023.
- [4] Atlassian. Scrum artifacts. <https://www.atlassian.com/agile/scrum/artifacts>, 2023.
- [5] Atlassian. Scrum roles. <https://www.atlassian.com/agile/scrum/roles>, 2023.
- [6] Mike Cohn. *Succeeding with Agile: Software development using Scrum*. Addison-Wesley, 2009.
- [7] ClickUp. Clickup docs: Product management and agile tools. <https://docs.clickup.com/en/articles/2095287-getting-started-with-clickup>, 2025.
- [8] Slack Technologies. Slack documentation. <https://slack.com/help/categories/200111606>, 2025.
- [9] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.
- [10] Joseph Redmon. Yolo: Real-time object detection. <https://pjreddie.com/darknet/yolo/>, 2023. Accessed: 2023-10-01.
- [11] Tesseract OCR Team. Tesseract ocr documentation. <https://github.com/tesseract-ocr/tesseract>, 2023.
- [12] J. Smith. Cost-benefit analysis of ocr in legacy medical systems. *Journal of Medical Technology*, 15(3):45–52, 2022.
- [13] Laravel Team. Laravel documentation. <https://laravel.com/docs>, 2023.
- [14] OpenCV Team. Opencv: Computer vision with python. <https://opencv.org>, 2023.
- [15] Google. Firebase cloud messaging performance guide. <https://firebase.google.com/docs/cloud-messaging>, 2023.